arXiv:2012.09276v2 [cs.LG] 5 Jan 2021

MEASURING DISENTANGLEMENT: A REVIEW OF METRICS

A PREPRINT

Julian Zaidi*
Ubisoft - La Forge
julian.zaidi@ubisoft.com

Jonathan Boilard
École de technologie supérieure
jboilard1994@gmail.com

Ghyslain Gagnon
École de technologie supérieure
ghyslain.gagnon@etsmtl.ca

Marc-André Carbonneau*
Ubisoft - La Forge
marc-andre.carbonneau2@ubisoft.com

January 7, 2021

ABSTRACT

Learning to disentangle and represent factors of variation in data is an important problem in AI. While many advances are made to learn these representations, it is still unclear how to quantify disentanglement. Several metrics exist, however little is known on their implicit assumptions, what they truly measure and their limits. As a result, it is difficult to interpret results when comparing different representations. In this work, we survey supervised disentanglement metrics and thoroughly analyze them. We propose a new taxonomy in which all metrics fall into one of three families: intervention-based, predictor-based and information-based. We conduct extensive experiments, where we isolate representation properties to compare all metrics on many aspects. From experiment results and analysis, we provide insights on relations between disentangled representation properties. Finally, we provide guidelines on how to measure disentanglement and report the results.

Keywords Representation Learning · Disentanglement · Metrics

1 Introduction

In recent years, learning disentangled representations has attracted a lot of attention from the machine learning community [1–25]. A disentangled representation independently captures true underlying factors that explain the data. Such representations offer many advantages: when used on downstream tasks, they improve predictive performance [15,16], reduce sample complexity [6, 18, 26, 27], offer interpretability [1, 26], improve fairness [17] and have been identified as a way to overcome *shortcut learning* in deep learning [28].

Originally, disentanglement was evaluated by visual inspection, but recent research efforts were devoted to propose metrics for more rigorous evaluations [1–13]. Most of the time, a new metric is proposed alongside a new representation learning method to highlight benefits not captured by existing metrics. Unfortunately, so far, it has often been unclear what these metrics quantify or not, and under which conditions [4, 9, 11, 16, 29]. Fair quantitative evaluation is important to assess research progress by comparing new representation learning methods with the state-of-the-art, but also equally important for practitioners when performing model selection and hyper-parameter tuning [12].

While most metrics correlate on simple data sets, they do not on more complex and realistic data [16]. Moreover, this correlation does not mean that they lead to the selection of the same model, as observed in [4, 16, 29]. We highlight this problem in our experiments in Section 5.1. Having metrics leading to different conclusions means that before choosing a model or a hyper-parameter setting, one must chose an appropriate metric for the application. This is not a

*Equal contribution

trivial task because existing metrics measure different properties of disentanglement and pose different, often implicit, assumptions on these properties. Moreover, these metrics are sometimes complex procedures themselves subject to hyper-parameter configuration. The goal of this paper is to provide some guidance to practitioners for selecting the appropriate metric for their application.

Very few papers discuss how to measure disentanglement. In [16], the authors conduct a large-scale study on disentanglement in the unsupervised setting. Their main conclusion is that disentangling predefined factors is impossible without inductive bias, and that random seeds and hyper-parameters have a greater impact on performance than the architecture of the studied models. They also conducted experiments to measure the degree of agreement of the six metrics used in the paper. They found that five of the six metrics correlate on the simple dSprites data set [1], but only mildly on other more realistic data sets. Unfortunately, no interpretations is given onto why one of the metrics sometimes inversely correlates with the others, or why metrics measuring different properties strongly correlate. In [5], the authors propose a framework for the evaluation of disentangled representations. They identify three desirable properties of a disentangled representation: explicitness, compactness and modularity. They introduce the idea that these properties should be quantified separately. They propose a new metric decomposed in three parts. The key idea of measuring different properties separately is also advocated in [6]. The authors point out that one of the three properties, compactness, is of lesser interest in practical scenarios. We will discuss these properties in detail in Section 2.

To our knowledge, [11] is the only study focusing on comparing metrics. The authors organize metrics based on the basic disentanglement properties they measure. Then, they verify that metrics assign a high score to all perfect representations and a low score to all representations that do not satisfy the measured property. Through demonstrations, they expose fail cases for several metrics. This constitutes a significant step towards the theoretical analysis of metrics in extreme cases.

In this paper, we propose an in-depth analysis of supervised disentanglement metrics with real-world applications in mind. We establish a clear taxonomy of metric families and underline their strengths and shortcomings. We compare the metrics with respect to many practical considerations such as robustness to noise and hidden factors, nonlinear relationship, accuracy, calibration and computational efficiency. We conduct experiments that abstract the representation learning model and data, which allows to generate representations for which we can accurately control and isolate the properties under study. Moreover, it also alleviates difficulties related to the identification of ground truth generative factors in data sets. We focus our analysis on *supervised* metrics (i.e. metrics that require ground truth factors) since there exist very few *unsupervised* metrics [9, 12, 13]. To our knowledge, this is the first time that such extensive and fully controlled experiments are conducted, and that metrics are compared in depth.

Contributions:

- We carry out an extensive review of disentanglement metrics, where we expose underlying assumptions, implementation complexity and other practical considerations.
- We establish a clear taxonomy of metric families and underline their strengths and short-comings.
- We conduct experiments that evacuate ambiguities introduced by learning algorithms and data sets to directly measure a metric's performance.
- We release our code to allow for the use of our metric implementations and the reproduction of our experiments[2].
- We provide recommendations for meaningful comparison between representations, as well as guidance for selecting appropriate metrics depending on the application context.

The rest of the paper is organized as follows: We start by identifying desirable representation properties that we wish to quantify. In Section 3, we define desirable characteristic for a metric. In Section 4, we survey existing metrics and present our taxonomy. In section 5, we present our experiments and their results. In Section 6, we discuss the implications of our findings and provide insight on how to measure disentanglement. We identify possible extensions to the paper in the conclusion.

[2] `https://github.com/ubisoft/ubisoft-laforge-DisentanglementMetrics`

2 Properties of a Disentangled Representation

Before analyzing metrics, we discuss what constitutes a disentangled representation. While there is no unanimously accepted definition of disentanglement, most agree on two main aspects [9, 14, 18, 26, 30, 31]: First, the representation has to be distributed. This means that an input is a composition of explanatory factors and corresponds to a single point in the representation space. Like in [5] we call this point a **code** in the remainder of this paper. Each factor is encoded in separate dimensions of the code. In Section 2.1, we further discuss factor independence and its implications. Second, the representation should also encode relevant information for the downstream task. Depending on the application and the representation learning algorithm, the way codes and factors relate to each other may vary significantly. We discuss Information content in Section 2.2.

2.1 Factor Independence in Representation

Factor independence means that the variation in one factor does not affect other factors, i.e. there is no causal effect between them [32]. In a disentangled representation factors are also independent in the representation space. In other words, a factor affects only a subset of the representation space, and only this factor affects this subspace. Most authors agree on the importance of this property which has different names (e.g., *disentanglement* [5], *modularity* [6]). In this paper, we use the name convention of [6] and refer to this property as **modularity**.

Some authors argue that the subset of the representation space affected by a factor should be as small as possible. Ideally, only one dimension completely describes a factor. This property is called *completeness* in [5], but is called *compactness* in [6]. In this paper we refer to this property as **compactness**.

The desirability of compactness relates to the type of factors for a given application. As argued in [6], enforcing compactness may be counterproductive. A group of code dimensions provides more flexibility when describing complex factors [33]. For instance, if an angle is represented as a single value $\theta \in [0, 2\pi]$, there is a discontinuity in the space at 2π. Alternatively, if two dimensions encode the angle (e.g. $\sin\theta$ and $\cos\theta$), continuity is preserved. Furthermore, sometimes factors are complex concepts like facial expression [19] or phonetic content [21], which are unlikely describable by a 1-dimensional space. As a second argument against enforcing strict compactness, Ridgeway and Mozer [6] explain that allowing redundancy in the learned latent space allows for different equivalent solutions, which facilitate model optimization from a practical standpoint.

All metrics assume that a set of independent factors exists in the problem. However, in practice, identifying useful and interpretable independent factors represents a challenging task [23]. Factors must be conceptually independent, but should also be statistically independent [2]. This condition is hard to satisfy in real-world data sets where certain factor realizations tend to co-occur more than others [14]. For example, in a data set of fruit images, we could be interested in two conceptually different factors: fruit type and color. However, color is statistically dependent with the fruit type. Dependent factors impact modularity score. If two factors were to share information, parts of the representation relate to both. The selection of factors is application-specific and is beyond the scope of this paper.

2.2 Information Content

To be truly useful, a representation should completely describe explanatory factors of interest. In other words, it should be possible to retrieve the complete factor realization from a point in the representation space (code). We call this property **explicitness** like in [14].

Perfect explicitness entails a generalizable relation between factors and codes was learned. The nature of this relation may vary and is implicitly assumed by some metrics. A linear relation between factors and codes is the simplest, and arguably most desirable, type of relation [6]. In applications like user-specified conditioned generation, learning a monotonic relation, even if not linear, allows for intuitive navigation in the representation space.

While a monotonic relation is a desirable characteristic for continuous factors, sometimes they are best described as categorical. In that case, the learning model must partition the representation space in regions corresponding to each category. The sample distribution in this space is multi-modal. Navigating this representation space by increasing the value of a code makes little sense. Measuring disentanglement with categorical factors necessitates metrics that do not make assumptions on the nature of the factor-code relation.

In [26], the authors argue that a good representation should be invariant to other factors and noise. Unfortunately, it is not always clear which factors are pertinent for the downstream tasks. This is why it is advocated to learn as many factors as possible and discard as little information as possible [19, 26].

3 Characterization of Disentanglement Metrics

Section 2 established three main properties of disentangled representations: modularity, compactness, and explicitness. This section enumerates desirable characteristics for a disentanglement metric.

Evidently, a metric should accurately measure a disentanglement property or a subset of the properties described in Section 2. Ideally the metric should not have failure modes as identified in [2, 11]. The scoring range of the metric should be calibrated. The metric should attribute the minimum score to a completely random or fully entangled representation and a perfect score to a perfectly disentangled representation. We verify this in the experiment of Section 5.2. We provide details on how to normalize the output range for metrics that were not normalized in their original implementation in Section 4.

In addition to being calibrated, a metric score should also evolve linearly with the quality of the disentanglement properties it measures. The worst-case scenario would be a metric that acts as a step function, which offers very low score interpretability and makes the comparison between two models with the same score meaningless. Moreover, such behavior renders the metric highly unstable. We evaluate how *linearly* the metric scores change with respect to explicitness in Section 5.2, as well as to compactness and modularity in Section 5.3.

As discussed in section 2.2, every metric makes an implicit assumption on the shape of the factor-code relationship. Sometimes, the application may dictate the type of relationship expected. For instance, if a linear relation is expected, metrics which penalize non-linear relations [5, 7] are best equipped to assess the quality of a representation. However, in most situations metrics should not make any assumptions and should capture nonlinear and multimodal relations. In Section 5.5 we compare metrics on monotonic and increasingly nonlinear relations.

A metric should not be overly sensitive to hyper-parameter configuration. Low parameter sensitivity ensures stability across different configurations. A metric overly sensitive to configuration behaves unpredictably and may lead to inaccurate conclusions when comparing models. Hyper-parameters can be predictor parameters (e.g., regularization weights), discretization granularity, batch size, validation protocol, etc. The best way to mitigate hyper-parameter sensitivity is to reduce the number of parameters to set in the first place.

In real-world applications, data sets are likely to be noisy. Metrics that measure compactness or modularity should be tolerant to noise, while explicitness metrics should reflect the amount of noise in the representation. We measure robustness to noise in the experiment of Section 5.2. In the context of measuring disentanglement, explaining factors that are not targeted by the metric can also be viewed as sources of noise. In most real-world applications, there will be several factors that will not be identified and measured. We evaluate tolerance to these distracting factors in Section 5.6.

Finally, there are practical considerations when choosing a metric. Some metrics have a high computational complexity, while others require a large number of data points to yield meaningful results. We briefly discuss these considerations in Section 6.2.

4 Metrics

This section surveys existing supervised metrics. We propose a new taxonomy that organizes metrics into three families. A family groups metrics based on their underlying working principle. **Intervention-based** metrics compare codes by creating subsets of data in which one or more factors are kept constant. **Predictor-based** metrics use regressors or classifiers to predict factors from codes. **Information-based** metrics leverage information theory principles, like mutual information (MI), to quantify factor-code relationships. Inspired by [11], we further divide each family in groups based on the disentanglement property that the metrics are designed to measure. Holistic methods capture two or more properties in a single score. Figure 1 shows all metrics organized following the proposed taxonomy. In the rest of this section, after introducing the notation, we go over all families in greater detail and describe metrics individually.

4.1 Notation

Inspired by [3], we denote a set of N observations ($\mathbf{x}$) as $X = \{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N\}$. Each of these observations is assumed to be completely explained by a set of M factors $\mathcal{V} = \{v_1, v_2, ..., v_M\}$ through a generative process $g(\mathbf{v}) \mapsto \mathbf{x}$. We denote $V = \{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_N\}$ the set of factor realizations that produced X. A representation learning algorithm is a mapping $r(\mathbf{x}) \mapsto \mathbf{z}$ where $\mathbf{z} \in \mathbb{R}^d$ is a point in the learned code space denoted by $\mathcal{Z} = \{z_1, z_2, ..., z_d\}$. $Z = \{\mathbf{z}_1, \mathbf{z}_2, ..., \mathbf{z}_N\}$ is the set of all points in X projected in the code space by $r(.)$. Supervised disentanglement metrics

Figure 1: Taxonomy of disentanglement metrics. Metrics are grouped in families based on their underlying working principle. Each family is divided in groups based on the disentanglement property that they are designed to measure.

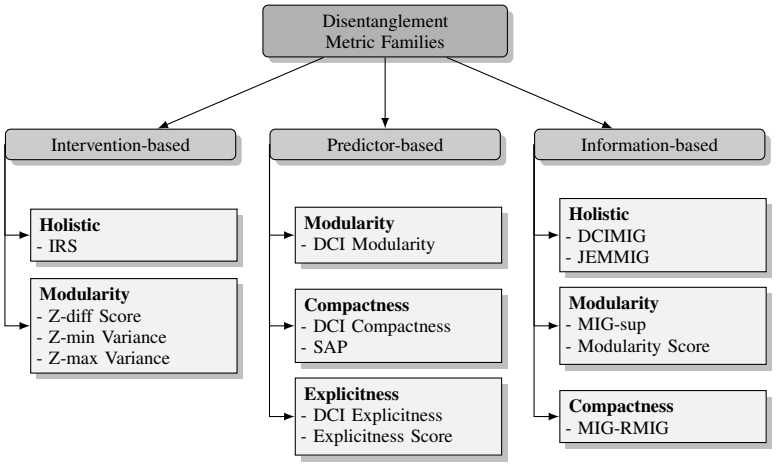
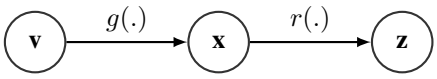


Figure 2: Illustration of the notation.



compute a score by comparing V to Z. Figure 2 illustrates the notation. Everywhere in the paper, boldface lowercase letters represent vectors.

4.2 Intervention-based Metrics

The metrics in this family evaluate disentanglement by fixing factors and creating subsets of data points. Codes and factors in the subsets are compared to produce a score. To sample the fixed size data subsets, these methods discretize the factor space. This sampling procedure necessitates large quantities of diverse data samples to produce a meaningful score. These metrics do not make any assumptions on the factor-code relations which is their main advantage. However, there are several hyper-parameters to adjust, such as the size and the number of data subsets, the discretization granularity, classifier hyper-parameters or the choice of a distance function. Finally, failure modes, situations where the metrics wrongly score representations, were identified in [2] and [11].

4.2.1 Z-diff

Z-diff metric [1], sometimes called the β-VAE metric, selects pairs of instances to create *batches*. In a batch, a factor v_i is chosen randomly. Then, a fixed number of pairs are formed with samples $\mathbf{v}^1$ and $\mathbf{v}^2$ that have the same value for the chosen factor ($v_i^1 = v_i^2$). Pairs are represented by the absolute difference of the codes associated with the samples ($\mathbf{p} = \left|\mathbf{z}^1 - \mathbf{z}^2\right|$). The intuition is that code dimensions associated with the fixed factor should have the same value, which means a smaller difference than the other code dimensions. The mean of all pair differences in the subset creates a point in a final training set. The process is repeated several times to constitute a sizable training set. Finally, a linear classifier is trained on the data set to predict which factor was fixed. The accuracy of the classifier is the Z-diff score. For a completely random classifier we expect an accuracy of $1/M$ where M is the number of factors. This can be used to scale the output closer to the $[0, 1]$ range.

4.2.2 Z-min Variance

Z-min Variance[3] metric [2], also called FactorVAE metric, was introduced to address some of the weaknesses of Z-diff metric. The intuition is the same as for Z-diff: code dimensions encoding a factor should be equal, if the factor value is the same. First, all codes are normalized by their standard deviation computed over the complete data set. For as subset, a factor is randomly selected and fixed at a random value. The subset contains sampled instances for which the selected factor is fixed at the selected value. Variance is computed over the normalized codes in the subset. The code

[3]We renamed the metric to avoid confusion with the model of the same name.

dimension with the lowest variance is associated to the fixed factor. Several subsets are created and the factor-code associations are used as data points in a majority vote classifier. Z-min Variance score is the mean accuracy of the classifier. Like for Z-diff, random classifier accuracy of $1/M$ can be used to scale the output closer to the $[0, 1]$ range.

4.2.3 Z-max Variance

Z-max Variance[3] metric [3], also known as R-FactorVAE, is similar to Z-min Variance. The main difference is the approach used to collect subsets of samples. Here, all factor values are fixed except one. This time, the intuition is that if all factors are the same except one, code dimensions corresponding to the free factor should exhibit higher variance. A majority vote classifier is also used to compute the score, but it is the code dimension with the highest variance that is chosen as a training point.

4.2.4 Interventional Robustness Score (IRS)

IRS [4] computes distances between sets of codes before and after an intervention on factor realizations. The intuition behind the metric is that changes in *nuisance* factors should not impact code dimensions attributed to targeted factors. First a reference set is created from instances where realizations of target factors are fixed. Then a second set contains instances with the same targeted factor realization, but different realizations of nuisance factors. The metric computes the distance (e.g. ℓ_2) between the mean of code dimensions associated to targeted factors. This sampling and distance measurement procedure is repeated several times and the maximum observed distance is reported. The final metric reports a weighted average of the maximum distances. The distances are weighted by the frequency of the factor realizations in the data set.

4.3 Predictor-based Metrics

These metrics train regressors or classifiers to predict factor realizations from codes ($f(\mathbf{z}) \mapsto \mathbf{v}$). Then the usefulness of each code dimension in predicting a given factor is analyzed. These methods are naturally suited to measure explicitness. They are typically equipped to deal with continuous factors as well as categorical factors simply by choosing an appropriate predictor. However, compared to Information-based metrics, they require more design choices and hyper-parameter tuning. This means a metric is more likely to behave differently from one implementation to another.

4.3.1 Disentanglement, Completeness and Informativeness (DCI)

In [5], the authors propose a complete framework to evaluate disentangled representations instead of a single metric. They report separate scores for modularity, compactness and explicitness, which they call disentanglement, completeness and informativeness. Regressors are trained to predict factors from codes. Modularity and compactness are estimated by inspecting the regressor's inner parameters to infer predictive importance weights R_{ij} for each factor and code dimension pair. In the paper, they use a linear lasso regressor or a random forest for non-linear factor-code mappings. For the lasso regressor, the importance weights R_{ij} are the magnitudes of the weights learned by the model, while the Gini importance [34] of code dimensions is used with random forests.

The compactness for factor v_i is given by $C_i = 1 + \sum_{j=1}^{d} p_{ij} \log_d p_{ij}$ where p_{ij} is the *probability* that code dimension z_j is important to predict v_i. These probabilities are obtained by dividing each importance weight by the sum of all importance weights related to this factor: $p_{ij} = R_{ij} / \sum_{k=1}^{d} R_{ik}$. The compactness of the whole representation is the average compactness over all factors.

Similarly, the modularity for code dimension z_j is given by $D_j = 1 + \sum_{i=1}^{M} p_{ij} \log_M p_{ij}$ where p_{ij} the is *probability* that code dimension z_j is important to predict only v_i. This time the importance weights are normalized with respect to codes: $p_{ij} = R_{ij} / \sum_{k=1}^{M} R_{kj}$. The modularity score for the whole representation is a weighted average of the individual code dimension modularity scores $\sum_{j=1}^{d} \rho_j D_j$. The scores are weighted by ρ_j to account for codes that are less important to predict factors. The weight ρ_j is the total importance for z_j normalized by the sum of all importance weights: $\rho_j = \sum_{i=1}^{M} R_{ij} / \sum_{k=1}^{d} \sum_{i=1}^{M} R_{ik}$.

The prediction error of the regressor measures the explicitness of the representation. With normalized inputs and outputs, it is possible to compute the estimation error for a completely random mapping and use it to normalize the score between 0 and 1. We postulate that a representation is not explicit if the mean squared error (MSE) of the predictor is higher than the expected MSE between two uniformally distributed random variables (X and Y). It can

be showed that MSE $= \mathbb{E}[(X-Y)^2] = 1/6$. Thus, explicitness can be written as $1 - 6 \cdot$ MSE. In our implementation, values under 0 are reported as 0.

4.3.2 Explicitness Score

In [6], the authors propose to use a classifier trained on the entire latent codes to predict factor classes, assuming that factors have discrete values. They suggest using a simple classifier such as logistic regression and report classification performance using the area under the ROC curve (ROCAUC). The final score is the average AUCROC over all classes for all factors. ROCAUC minimal value is 0.5 which means that the score needs to be normalized to obtain a value between 0 and 1. In our implementation we balance weights in the loss of the logistic regression to account for class imbalance.

4.3.3 Attribute Predictability Score (SAP)

SAP [7] attributes a score S_{ij} to all pairs of factor v_i and code dimension z_j. A linear regression predicts a continuous factor from each code and S_{ij} is the R^2 score of the regression. For categorical factors, it fits a decision tree on codes and reports balanced classification accuracy. Scores corresponding to codes with energy below a user specified threshold (i.e. *dead-codes*) are set to 0. The final SAP score is obtained by computing the difference between the two highest S_{ij} for all factors:

$$\text{SAP} = \frac{1}{M} \sum_{i}^{M} S_{i\star} - S_{i\circ} \tag{1}$$

In this equation, $S_{i\star}$ is the highest score for factor v_i, while $S_{i\circ}$ is the second highest. M is the number of factors. Similar $S_{i\star}$ and $S_{i\circ}$ means that explicitness is low if both values are low. Two similarly high values indicate that more than one code dimension encode the factor which means low compactness. This corresponds to the *gap* idea in MIG (Section 4.4.1)

4.4 Information-based Metrics

Information-based metrics compute a disentanglement score by estimating the mutual information (MI) between the factors and the codes. These methods require fewer hyper-parameters than intervention-based and predictor-based metrics. Moreover, they do not make assumptions on the nature of the factor-code relations.

However, while elegant in theory, in practice the estimation of entropy and MI is non-trivial. Even assessing the quality of the estimators remains an open problem [35]. It requires quantization of both spaces or a sampling procedure which needs to be parameterized. Most existing public MI-based metric implementations use the maximum likelihood estimator. For example in the widely used disentanglement_lib[4] MI is computed as follows:

$$I(v, z) = \sum_{i=1}^{B_v} \sum_{j=1}^{B_z} P(i, j) \log \left(\frac{P(i, j)}{P(i)P(j)} \right) \tag{2}$$

Factor and code spaces are discretized in B_v and B_z bins. $P(i)$ and $P(j)$ are estimated as the proportion of samples assigned to bin i and j respectively over all samples (N). Similarly $P(i, j)$ is the proportion of samples assigned to both bin i and j. Problems arise when estimating from under-sampled high dimensional data [36]. This is the case when computing MI, or joint entropy, between a factor and more than one dimension of the latent space. Moreover, the estimated MI value is affected by the granularity of the discretization which makes the metrics sensitive to this parameter.

4.4.1 Mutual Information Gap (MIG)

MIG [8] computes the MI between each code and factor $I(v_i, z_j)$. Then, the code dimension with maximum MI is identified $I(v_i, z_\star)$ for each factor. After, the second highest MI, $I(v_i, z_\circ)$, is subtracted from this maximal value. This difference constitutes the *gap*. The gap is then normalized with respect to the total MI associated with the factor:

$$\text{MIG} = \frac{I(v_i, z_\star) - I(v_i, z_\circ)}{\sum_{j=1}^{d} I(v_i, z_j)} \tag{3}$$

[4] https://github.com/google-research/disentanglement_lib

The MIG score of all factors are averaged to report one score.

Robust MIG (RMIG) was proposed in [9]. It is identical to MIG in essence, but proposes a more robust formulation when MI is computed from the input space, which does not apply in our context.

4.4.2 Joint Entropy Minus Mutual Information Gap (JEMMIG)

MIG verifies that the information related to a factor is expressed by only one code dimension (compactness). However, modularity is not directly measured. For instance a code dimension could contain information about more than one factor. JEMMIG [9] addresses this drawback by including the joint entropy of the factor and its best code.

$$\text{JEMMIG} = H(v_i, z_\star) - I(v_i, z_\star) + I(v_i, z_\circ) \tag{4}$$

As opposed to MIG, this metric indicates a high disentanglement quality with a lower score. The maximum value is bounded by $H(v_i) + \log(B_z)$, where B_z is the number of bins used in the code space discretization. This means that JEMMIG can be rewritten as follows to get a score between 0 and 1:

$$\widehat{\text{JEMMIG}} = 1 - \frac{H(v_i, z_\star) - I(v_i, z_\star) + I(v_i, z_\circ)}{H(v_i) + \log(B_z)} \tag{5}$$

As done for MIG, JEMMIG is reported as the average for all factors v_i.

4.4.3 MIG-sup

MIG-sup [10] is an extension of MIG. Like JEMMIG, it addresses the fact that MIG measures compactness, but does not measure modularity. It is designed to be used in conjunction with MIG. The idea is similar to MIG except that the MI gap is computed from the code point-of-view:

$$\text{MIG-sup} = I(z_j, v_\star) - I(z_j, v_\circ) \tag{6}$$

$v_\star$ is the factor that has the highest MI with code dimension z_j. $v_\circ$ is the factor that has the second highest MI with code dimension z_j. $I(z_j, v_i)$ is the MI normalized by the entropy of the factor v_i. MIG-sup is reported as the average gap over all code dimensions.

4.4.4 Modularity Score

To measure modularity, in [6] the factor $v_\star$ which shares the maximum MI for each code dimension z_j is identified. This maximal MI value $I(v_\star, z_j)$ is then compared with MI values of all other factors:

$$\text{modularity} = 1 - \frac{\sum_{i \in \mathcal{V}_{\neq\star}} I(i, z_j)^2}{I(v_\star, z_j)^2 (M-1)} \tag{7}$$

We denote $\mathcal{V}_{\neq\star}$ as the set of all factors except $v_\star$ and M is the number of factors. The average modularity score over all codes is reported.

4.4.5 DCIMIG

DCIMIG [11] is a metric inspired by DCI and MIG. Like MIG, it computes MI gaps between factors and code dimensions. Like DCI it analyzes a factor-code importance matrix. However, unlike DCI, DCIMIG reports a single score for all three disentanglement properties. DCIMIG starts by computing the MI between each factor and code dimension $I(v_i, z_j)$. Then, the factor with maximum MI, $I(v_\star, z_j)$, is identified for each code. After, the second highest MI, $I(v_\circ, z_j)$, is subtracted from this maximal value. Thus we obtain a *gap* for each code dimension $R_j = I(v_\star, z_j) - I(v_\circ, z_j)$. Each of these gaps R_j relates to a code dimension and the factor for which MI is maximal. For each factor v_i, we find all associated gaps R_j and use them as score S_i for this factor. If there are more than one R_j associated with the factor, S_i equals the highest R_j. If there are none, $S_i = 0$. Finally the metric is the sum of all scores normalized by the total factor entropy:

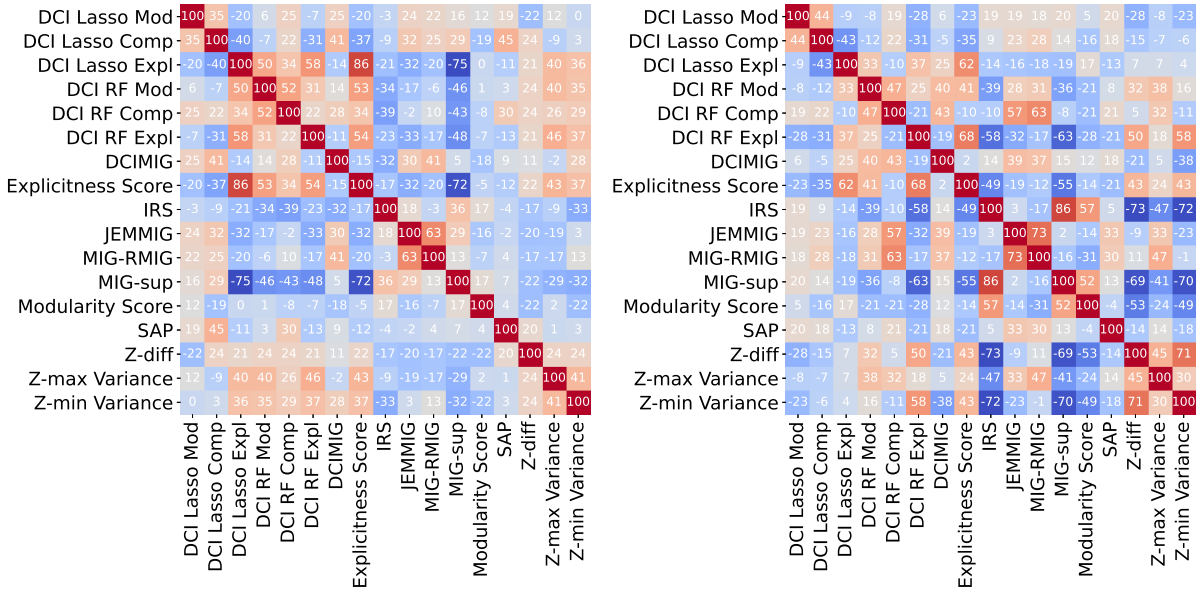
$$\text{DCIMIG} = \frac{\sum_{i=1}^{M} S_i}{\sum_{i=1}^{M} H(v_i)} \tag{8}$$

5 Experiments

In the experiments, we abstract the relation $\mathbf{z} = r(g(\mathbf{v}))$ by $\mathbf{z} = f(\mathbf{v})$. Except for Section 5.1, we do not learn representations on data sets, but instead we directly define $f(\mathbf{v})$ as a function that allows for complete control over the parameters of the representation evaluated by the metrics. This removes any ambiguities related to the quality of the data set, the choice of learning algorithm, its training and the choice of factors to disentangle. We assume factors are selected following $v_i \perp\!\!\!\perp v_j, i \neq j$ as prescribed in [4]. The code for the experiments is available at `https://github.com/ubisoft/ubisoft-laforge-DisentanglementMetrics`

5.1 Model Selection

In this experiment we verify if using different metrics to perform model selection or hyper-parameter tuning leads to different outcomes. Here, knowing the exact factor-code relation is unimportant. Therefore, we learn representations from data sets to mimic a situation a practitioner might encounter. We optimize two hyper-parameters for β-VAE [1]: The regularization strength ($\beta \in \{0.001, 0.01, 0.1, 1, 10, 100\}$) and the dimensionality of the representation space ($d \in \{2, 4, 8, 16, 32, 64\}$). We use two standard data sets Cars3D [37] and SmallNORB [38]. For each configuration, we train the model for 300k training steps using the Adam optimizer and a batch size of 64 similarly to [16]. Then, we evaluate the learned representation with every metric. For each metric, we rank the hyper-parameter configurations. We measure the agreement between all metrics using the Kendall rank correlation coefficient [39] and report results in Figure 3.



(a) Cars3D (b) SmallNORB

Figure 3: Kendall rank correlation coefficient ($\times 100$) between metrics for model rankings.

We find that different metrics lead to different choices of models, which is consistent with results found in [4, 16, 29]. While some metrics correlate, most of the time the correlation is weak and worse, often inverse. This confirms that these metrics are not equivalent and measure different properties under different assumptions, which motivates the present study. It would be hazardous to interpret these correlations between metrics because we do not know the exact factor-code relations, which is why we devised fully parameterized relations for the subsequent experiments.

5.2 Perfect Disentangled Representation with Noise

In this section we evaluate how metrics behave in a scenario where we gradually depart from a perfect disentangled representation to a completely random representation. In a perfect representation, factors completely describe the data and have a one-to-one relation with codes. This scenario shows how metrics behave as explicitness decreases under perfect compactness and modularity. We also verify metrics are well calibrated (i.e. attribute a perfect score to perfect representation and a low score to noise). The factor-code relation is defined by:

$$\mathbf{z} = f(\mathbf{v}) = (1-\alpha)\mathbf{v} + \alpha\mathbf{n} \tag{9}$$

where $\mathbf{n} \sim \mathcal{U}(0,1)$, $\alpha \in [0,1]$ and $\mathbf{v}, \mathbf{z} \in \mathbb{R}^{M=d}$. We simulate a problem with 8 factors ($d = M = 8$). We tried different number of factors and found conclusions to be similar. Given a set of factor realizations V we use $f(.)$ to obtain its representation in the code space Z. The set V contains $N = 20k$ samples from the uniform distribution. We use the same 20k samples for all metrics. When necessary, factor values are discretized into 10 equal bins. We evaluate α at $\{0.0, 0.2, 0.4, ..., 1.0\}$. We repeat the experiment with 100 different sampled versions of V using 100 random seeds and report the average result. Figure 4 shows the mean score for all metrics as the noise level (α) increases.

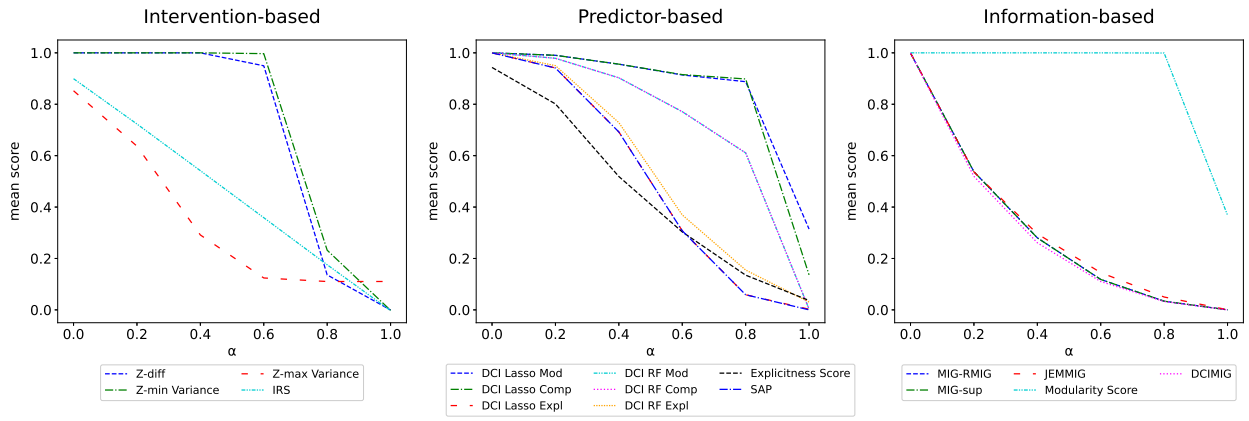


Figure 4: Metric scores for perfectly disentangled representations under increasing noise level (α).

Most metrics recognize a perfect representation and attribute a perfect score. There are three exceptions. IRS is unlikely to produce a perfect score for any representation because it computes a distance between codes for factors that are *binned* together. Factors in the same bin are likely to differ within the range of the bin, which in turn results in small distances in code values for the same factor bin. This explains why IRS cannot attribute a perfect score to a perfect representation. To circumvent this problem, smaller discretization needs to be applied if the number of samples is large enough for the given application. The Explicitness score is the average of ROCAUC for $M \times 10 = 80$ logistic regression classifiers trained in a one-versus-the-rest strategy. One classifier is trained for each bin value per factor. The optimizer does not consistently find the optimal solution for all classifiers which leads to an ROCAUC under 1. Z-max Variance requires a dense combination of factor values to sample meaningful batches for the majority vote classifier. The 20k examples used in the experiment, when discretized in 10 bins, do not provide enough examples for a same factors realization. The direct consequence is a biased estimation of the variance, which causes a score under 1 for a perfect representation and a score higher than 0 for a completely random representation. To circumvent this problem, coarser discretization needs to be applied, which in turn might lead to an overestimation of the scores.

The majority of metrics attribute a score near 0 to complete noise. However, DCI for modularity and compactness scores the representation over 0.3 when using a lasso regressor. Even if the regressor accuracy is low, weights are still learned and compared to compute compactness and modularity scores. The regularization term in lasso pushes some weights towards 0 and thus sizable differences between them will be observed. This leads to observing random isolated factor-code relations which drive the score up. When using the Modularity score, the MI between each factor and code dimension should be similar. However, maximal MI value normalizes the score, which leads to a wrongfully optimistic value in most experiments in this paper.

When measuring explicitness under noise, an ideal metric score should steadily decrease as the noise level increases. IRS is a perfect example of a score that decreases linearly with noise. In fact, most metrics that focus on explicitness perform adequately. If explicitness metric scores should decrease in the presence of noise, we expect a different behavior from modularity or compactness metrics. Ideally, a metric should recognize these disentanglement properties, even in noisy representations. The predictor-based DCI exhibits a high noise robustness, which makes sense since predictors naturally discard noise information to improve generalization. This being said, their tendency to observe random isolated factor-code relations discussed above inflates this perception of noise robustness. In addition to being well calibrated, intervention-based metrics Z-diff and Z-min Variance also proved to be quite robust. Inversely, this experiment exposes the vulnerability to noise of information-based metrics. Noise causes codes to be assigned to neighbouring bins which decreases the observed MI between factors and codes.

5.3 Decreasing Compactness and Modularity

In the previous section we observed how metrics behave as explicitness decreases. Now, we study what happens as we gradually decrease compactness and modularity, while explicitness remains perfect. The embedding function is constructed given by $\mathbf{z} = f(\mathbf{v}) = \mathbf{v}R$. The projection matrix R is defined by:

$$R = \begin{bmatrix} 1-\alpha & \alpha & 0 & \cdots & 0 \\ 0 & 1-\alpha & \alpha & \cdots & 0 \\ 0 & 0 & 1-\alpha & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha & 0 & 0 & \cdots & 1-\alpha \end{bmatrix}$$

When $\alpha = 0$, R is the identity matrix and the representation is perfectly compact and modular. As α increases, all factors are represented by two code dimensions and each code dimension relates to two factors. Figure 5 shows how metric scores evolve as the representation becomes less modular and less compact.

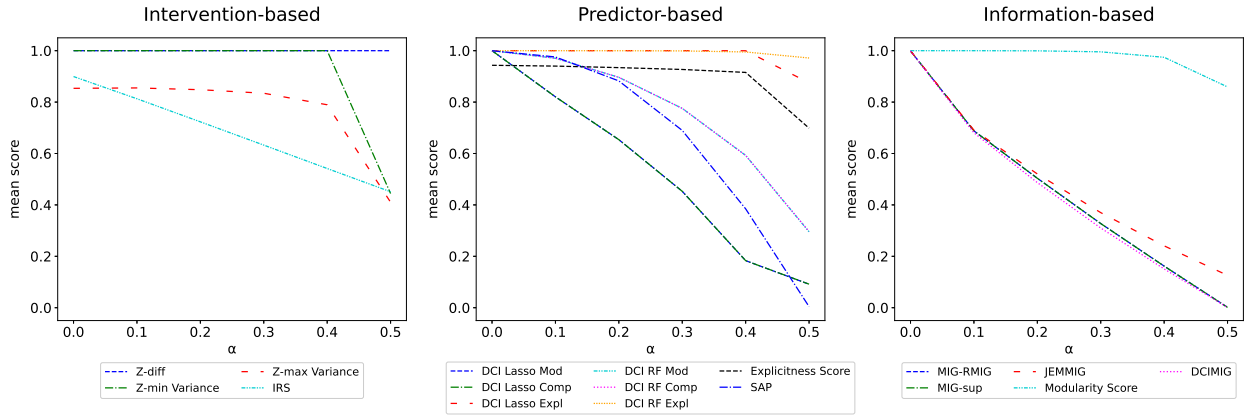


Figure 5: Mean metric scores as the representation becomes less modular and less compact.

Results from this experiment reveal several differences amongst metrics. Since the representation allows for complete recovery factor values, explicitness metrics maintain a high score as expected. We would expect modularity and compactness metric scores to linearly decrease as α increases. This is the case for most information-based metrics and predictor-based metrics. Interestingly, some metrics output a 0 score when a code dimension relates to two factors or *vice versa*. This means that these metrics make no distinction between a representation where a code dimension relates to two factors and a representation where a code dimension relates to all factors. DCI is the best equipped metric to quantify this distinction because it will never yield a zero score unless all codes are equally relevant to predict the factors. The experiment also reveals a failure mode of intervention-based metrics as identified in [11]. These metrics consistently attribute a high score to the representations even when it is imperfect. The worst case is Z-diff that attributes a perfect score even when $\alpha = 0.5$. It is always trivial for the classifier to identify a factor by finding the distinct combination of two code dimensions with to lowest difference.

5.4 Modular but not Compact

Here we evaluate how the metrics behave when the representation is perfectly explicit and modular but not compact. As discussed in Section 2.1, compactness is of lesser interest than modularity in many real-world applications. Thus, it is important to assess the ability of the metrics to recognize modularity even when several code dimensions are used to describe a single factor.

The first experiment of this section emulates a model that has learned a decomposed representation of angles. When a scalar defines an angle, the representation space has a discontinuity at 2π. Decomposing angles in sine and cosine values ensures the space is continuous which is preferred in many applications. Here, each factor represents an angle $\theta \in [0, 2\pi[$. Codes represent angles as $\cos\theta$ and $\sin\theta$. Factor realizations define four angles: $\mathbf{v} = [\theta_1, \theta_2, \theta_3, \theta_4]$ and the corresponding codes are given by $\mathbf{z} = [\cos\theta_1, \sin\theta_1, \cos\theta_2, ..., \sin\theta_4]$. Factor values are discretized to 10 bins ($v_i \in \{0, \pi/5, 2\pi/5, ..., 9\pi/5\}$).

Table 1: Scores attributed to disentanglement where a factor is encoded with more that one code.

	Z-diff	Z-min Variance	Z-max Variance	IRS	DCI Lasso Modularity	DCI Lasso Compactness	DCI Lasso Explicitness	DCI RF Modularity	DCI RF Compactness	DCI RF Explicitness	Explicitness Score	SAP	MIG-RMIG	MIG-sup	JEMMIG	Modularity Score	DCIMIG
$\theta \to [\cos\theta, \sin\theta]$	1.0	1.0	1.0	0.8	0.8	1.0	0.6	1.0	0.7	1.0	1.0	0.6	0.0	0.7	0.4	1.0	0.6
$\theta \to [\theta, \theta]$	1.0	1.0	1.0	0.9	1.0	1.0	1.0	1.0	0.7	1.0	1.0	0.0	0.0	1.0	0.5	1.0	1.0
$\theta \to [\theta, \theta, \theta, \theta]$	1.0	1.0	1.0	0.9	1.0	1.0	1.0	1.0	0.4	1.0	1.0	0.0	0.0	1.0	0.5	1.0	1.0

Following the same idea, we create a second data set where factors are encoded by two code dimensions. However, this time, factor-code relations are linear. This corresponds to a scenario where the representation learning algorithm has learned redundant codes. This scenario allows for comparison of results obtained in the previous experiment without having to account for the nonlinear relations (sine and cosine). We keep the same four factors, but we use linear relations: $\mathbf{v} = [\theta_1, \theta_2, \theta_3, \theta_4]$ corresponds to $\mathbf{z} = [\theta_1, \theta_1, \theta_2, ..., \theta_4]$.

Finally, we repeat the same experiment except that there are only two factors associated with four code dimensions each: $\mathbf{v} = [\theta_1, \theta_2]$ corresponds to $\mathbf{z} = [\theta_1, \theta_1, \theta_1, ..., \theta_2]$. Following the sampling methodology described in Section 5.2, we apply the metrics and report scores in Table 1.

On the left of the result table, we can see that none of the intervention-based metrics penalizes representations for not being compact. This is in accordance with the results from the previous experiment. Predictor-based metrics exhibit different behaviors depending on the type of predictor used. The lasso predictor, unsurprisingly, has trouble dealing with the nonlinear sine and cosine relations. More interestingly, it has problems dealing with redundant codes. Since only one code dimension is necessary to predict a factor, the information from the duplicated code dimensions is discarded, encouraged by the regularization term. This falsely leads the metric to think that only one code dimension is associated with the factor, hence the perfect compactness for all experiments. Using a random forest predictor overcomes this problem. As observed in the preceding experiment, SAP and MIG which measure compactness cannot express to what degree a representation is not compact. This is because they compute a *gap* which subtracts the two most significant terms and ignores all of the others. The same can be said for JEMMIG. DCIMIG while intended as a holistic method does not penalize non-compactness in this experiment. Finally, we can observe that information-based metrics have trouble dealing with nonlinear relations. This will be discussed in greater detail in the next experiment.

5.5 Nonlinear relations

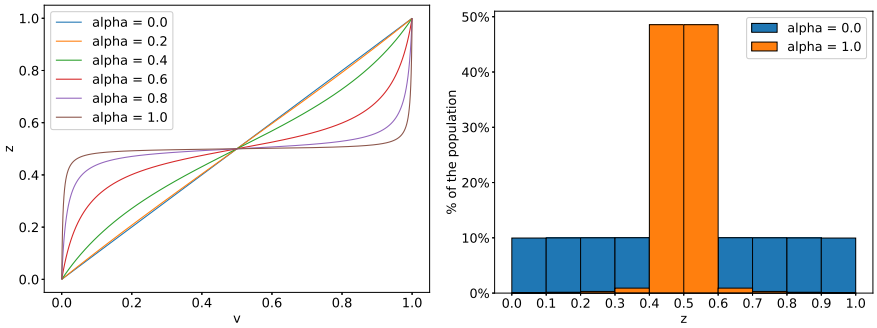
Here we explore representations with nonlinear relations between factors and codes. The representation is kept perfectly compact and modular and should receive a perfect score from all metrics. The mapping function becomes increasingly nonlinear as α increases, but is always monotonic for $v \in [0, 1]$:

$$\mathbf{z} = f(\mathbf{v}) = 1000^{-\alpha+0.25} \tan(\omega(\mathbf{v} - 0.5)) + 0.5 \tag{10}$$

where $\omega = 2\arctan(1000^{\alpha-0.25}/2)$. When α = 0, the relation is practically linear, and when α is 1 the relation takes the shape of a tangent function as shown in Figure 6a. This relation is interesting because it highlights potential problems with using a linear regressor to compute scores, as well as potential problems inherent to discretization. Results are reported in Figure 7.

As expected, predictor-based metrics using a linear regression to measure explicitness, DCI lasso and SAP, underperform as the factor-code relation becomes less linear. The monotonic nature of the relation allows DCI lasso to accurately score modularity and compactness. Naturally, a more expressive predictor makes the metric robust to more complex relationships.

This experiment highlights potential problems with discretization which is at the center of information-based metrics, as well as intervention-based metrics and even some predictor-based metrics like the Explicitness score. Equal binning of the code space results in a larger amount of the population being assigned to the middle bins. Figure 6b shows the proportion of samples assigned to each discretization bins when $\alpha = 0$ and $\alpha = 1$. This uneven population



(a) Shape of the factor-code relation as parameter α increases.

(b) Effect of nonlinearity on discretization bins population.

Figure 6: Shape of the parametric factor-code relation and its effect on discretization

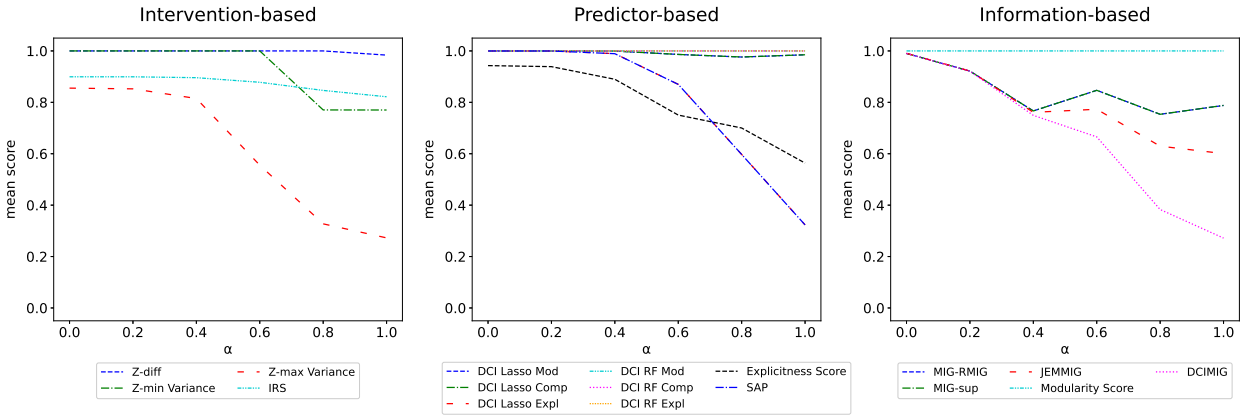


Figure 7: Metric scores for perfectly disentangled representations with increasingly nonlinear factor-code relation (α).

distribution lowers the code space entropy and in turn affects MI computation. Similarly, it affects how subsets are created in intervention-based metrics. This explains why a large proportion of metrics fail to properly score the perfectly modular, compact and explicit representation.

5.6 When Factors Partially Describe Data

This experiment simulates the case where metrics measure only a fraction of all the generative factors. This frequently happens in real-world scenarios because it is difficult to identify all generative factors in a data set. For instance, channel noise may corrupt data and get modeled in some dimensions of the code like in [40]. These non-measured factors still need to be encoded to preserve explicitness and because they can be useful for downstream tasks.

From the metric point-of-view, code dimensions corresponding to non-measured factors are seen as noise or dead-codes [5] which affects metric scoring. We generate perfect representations in the same way as in Section 5.2 but without noise ($\alpha = 0$). The relation between factors and codes becomes the identity $\mathbf{z} = f(\mathbf{v}) = \mathbf{v}$. Then, we apply the metrics to these perfect representations and vary the proportion of measured factors.

When metrics measure all of the 8 generative factors captured by the perfect representation, their score should be maximal. Metrics should maintain that maximal value as the proportion of measured factors decreases because the representation does not change. Figure 8 shows how metric scores evolve as the proportion of measured factors decreases.

Most metrics are equipped to deal with non measured factors, except for Z-max Variance, IRS, MIG-sup and the Modularity score. This limits their relevance in contexts outside of academic toy problems. Successful methods that measure disentanglement from the code point-of-view implement a mechanism to discard *dead-codes* [5]. A dead-code is a code dimension that does not inform on any factor. MIG-sup and the Modularity score do not provide such

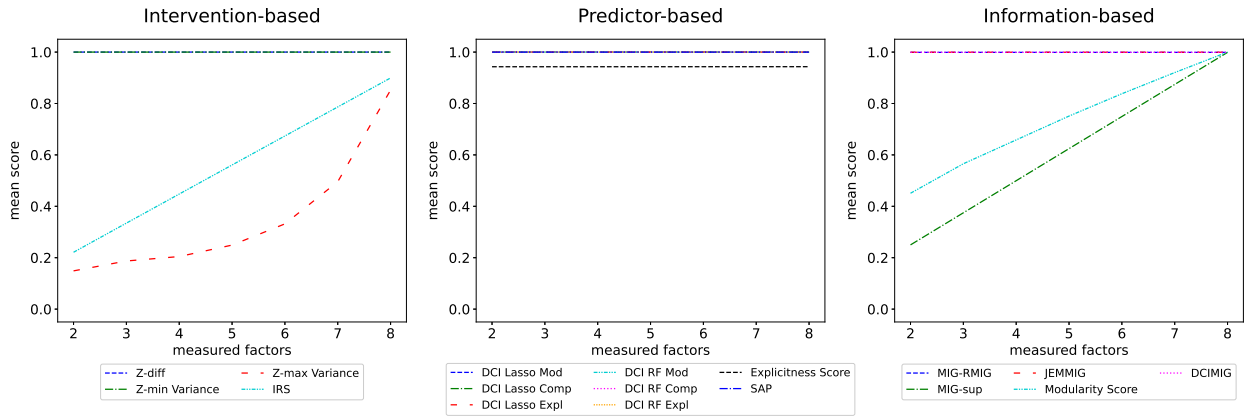


Figure 8: Scores for perfectly disentangled representations. The abscissa indicates how many of the 8 factors are measured by the metrics.

mechanisms. When sampling to create subsets, IRS and Z-max Variance implicitly assume that when all known factors are fixed, corresponding codes are also fixed. This assumption is violated when there are other sources of variation for the code than the known factors.

6 Discussion

This section summarizes our learnings from the experiment results and insights from relevant papers. After discussing relations between representation properties, we identify best practices for measuring disentanglement in real-world applications. Finally, we provide recommendations on how measurements should be reported.

6.1 Relations Between Representation Properties

While disentanglement properties can be measured separately, they are implicitly linked together. This makes the analysis of disentanglement more difficult and might have motivated the holistic approach of some metrics.

Evidently, some degree of explicitness is necessary to observe modularity or compactness otherwise it would not be relevant to compare factor-code relations. However, as shown in the experiment of Section 5.2, a high level of modularity and compactness can be observed, even when explicitness is minimal. In other words, **explicitness is a necessary condition for modularity and compactness, but the magnitude of these properties does not inform on the magnitude of the explicitness.**

Modularity and compactness are linked together by the size of the code space. When all factors are represented, if the code space is the same dimensionality as the number of factors, perfect modularity necessarily implies perfect compactness. This relation is not symmetric. Perfect compactness does not necessarily means perfect modularity in this situation. A code dimension could encode two factors even if each factor is encoded by only one dimension. This would however mean that there are one or more dead-codes. As the code space increases, under perfect modularity, imperfect compactness is possible. The ratio between the code space dimensionality and the number of factors determines how much compactness is allowed to deteriorate. As a general rule, the code space should be larger than the factor space to allow for composite factors and non-measured factors. The fact that the code space size links compactness and modularity could explain correlations sometimes observed between metrics that focus on only one of these properties like in [16].

Modularity is more important in practice than compactness. This has already been observed in [14]. There are several reasons why researchers and practitioners should focus on modularity instead of compactness. The main reason is that measuring compactness is desirable only if one can identify *atomic* (1D) factors, which is often difficult or impossible in real-world applications. As mentioned earlier, some basic concepts like angle or color are best represented in a 2D or 3D space. In addition, any composite factors (e.g., object type in images or speaker identity in speech segments) are more meaningfully represented in a multi-dimensional space, where each dimension represents an atomic factor. These atomic factors are sometimes concepts difficult to identify, describe and measure. For example, if one wants to disentangle speaker identity from speech segments. Many atomic factors define a voice print. Some are simpler to identify and measure like pitch and speech rate, but they do not paint the whole picture. The complete

Table 2: Summary of findings from experiments and analysis. For a metric to possess a desired characteristic (✓), it has to be true in theory, as well as in practice. The robustness to noise characteristic does not apply to explicitness metrics.

Metric	Modularity	Compactness	Explicitness	Calibrated	Robust to Noise	Robust to non-measured factors	Nonlinear Relation	Discretization-free	Few Hyper-parameters
Z-diff [1]	✗	✗	✗	✓	✓	✓	✗	✗	✗
Z-min Variance [2]	✗	✗	✗	✓	✓	✓	✗	✗	✗
Z-max Variance [3]	✗	✗	✗	✗	✗	✗	✗	✗	✗
IRS [4]	✓	✗	✓	✗	n/a	✗	✗	✗	✗
DCI - Lasso [5]	✓	✗	✓	✗	✓	✓	✗	✓	✓
DCI - Random Forest [5]	✓	✓	✓	✓	✓	✓	✓	✓	✗
Explicitness Score [6]	✗	✗	✓	✗	n/a	✓	✗	✗	✓
SAP [7]	✗	✓	✓	✓	n/a	✓	✗	✓	✓
MIG-RMIG [8, 9]	✗	✓	✗	✓	✗	✓	✗	✗	✓
MIG-sup [10]	✓	✗	✗	✓	✗	✗	✗	✗	✓
JEMMIG [9]	✓	✓	✓	✓	✗	✓	✗	✗	✓
Modularity Score [6]	✗	✗	✗	✗	✓	✗	✗	✗	✓
DCIMIG [11]	✓	✗	✓	✓	✗	✓	✗	✗	✓

set of atomic factors for voice print remains elusive, even for speech experts. Nonetheless, they must be encoded to successfully perform a downstream task like speaker identification or conditioned speech synthesis. the same goes for illumination in a picture. In practice, one might want to isolate the effect of a light source. However, light sources have many attributes like, 3D position, color, shape, size, intensity, etc. All these factors have to be identified and quantified to measure compactness. For both these example applications, useful disentanglement is best measured through modularity. A high modularity score indicates that all atomic factors of interest are contained in a defined subset of the code space.

The factor space and the code space need to be aligned to accurately measure disentanglement. Perfect compactness and modularity entails complete disentanglement of generative factors. However, it is possible to learn a representation where factors are completely disentangled, and yet measure low compactness and modularity scores because of a misalignment between space axes. As a thought experiment, if we take a perfectly disentangled representation, where each factor corresponds to only one code, and rotate this representation space around any axis. The resulting rotated representation space maintains the independence between factors. However existing metrics will fail to capture perfect modularity or compactness because variations in one factor will cause variations on several dimensions of the code space and *vice versa*. We believe a metric should be robust to this kind of misalignment and be able to evaluate representations by looking at them from the right "point-of-view". In practice, axis alignment can be enforced during learning through supervision, or indirectly encouraged like in VAEs [22,41], but cannot be guaranteed in most unsupervised learning settings [16].

6.2 Practical Considerations for Choosing a Metric

In this section, we extract conclusions from our analysis and experimental results summarized in Table 2. We provide guidance for choosing an appropriate metric for real-world application, and we highlight practical considerations when measuring disentanglement.

Metrics that do not account for non-measured factors should be avoided in real-world scenarios. As discussed in Section 2.1, in practice, identifying factors is a challenging task. Identifying all factors is even more difficult. Moreover, when identified, factors must be measured which is sometimes impossible. This means that for most applications, there will exist unidentified factors explaining the data, which will cause some metrics to underestimate modularity as shown in Section 5.6.

Using discretization is not trivial and has an impact on score. As we saw in Section 5.5, discretization of the code and the factor space has considerable impact on the ability of metrics to deal with nonlinear relations. The granularity of the discretization has an impact on the estimated MI, which is the centerpiece of information-based metrics. Intuitively, MI informs on how easy it is to predict a variable A knowing B. Suppose A is a random variable and $B = A + \sigma$ where σ is random noise. On one extreme, if both variables are discretized in 1 bin, then the MI is maximum. On the other end of the spectrum, A and B are discretized in a large number of narrow bins. If the number of samples is limited, it is unlikely that B will help predict the exact bin of A. In that case, MI will appear to be low even if there exists a strong relation between A and B. In intervention-based metrics, the discretization granularity determines the degree of similarity/dissimilarity of examples grouped in the same subset. A too coarse discretization creates heterogeneous groups that are considered homogeneous, which biases results. A too fine discretization makes it impossible to create large enough subsets of data points with the same fixed value. To our knowledge, no procedure has been proposed yet to strike the right balance between coarse and fine discretization for any type of metric.

Table 2 compiles our experimentation results and analysis. For a metric to possess a characteristic (✓), it has to be true by design and not disproven experimentally. For instance, DCI with lasso regressor is marked with (✗) because it has a failure mode when measuring compactness as shown in Section 5.4, even if in theory it can measure the property. Same goes for metrics necessitating discretization when dealing with nonlinear relations.

DCI implemented with random forest is the best all around metric. Measuring disentanglement properties separately allows for accurate scoring. Because random forest is an expressive model, it can discover nonlinear relationships and does not suffer from problems related to discretization. Moreover, random forests can be used as classifiers and regressors which makes them appropriate for applications mixing continuous and categorical factors. DCI implements a weighting scheme that accounts for dead-codes in problems where not all factors can be identified. However, there are two important disadvantages to DCI. First, modeling relations with random forests requires a bit of expertise to set the hyper-parameters and determine a relevant criterion for code dimension importance. **The hyper-parameters must be tuned using an appropriate cross-validation procedure, to ensure proper regularization of the model.** Otherwise it will overfit, which results in an overestimation of explicitness as well as an underestimation of modularity and compactness. This cross-validation procedure is time consuming which is the second main disadvantage of the method. In fact, DCI with random forest is the most computationally expensive of all metrics studied in this paper.

In their current state, metrics in the intervention-based family should be used with great caution. They require large quantities of data to create subsets with fixed values. This prohibits their application in problems with limited quantities of data with labeled factors. They are subject to vulnerabilities associated with discretization. Moreover, they are prone to failure modes, which limits their reliability. Finally, unlike most metrics from other families, they do not produce a factor-code relation matrix, which makes their results difficult to interpret and less helpful when debugging.

Information-based metrics are in theory flexible and elegant. They can measure factor-code relations of any shape, continuous or categorical, with a minimal amount of hyper-parameter tuning. However, the aforementioned challenges with discretization limit their universality and makes them vulnerable to noise. Also metrics based on information *gaps* like MIG, only consider the difference between the two best candidates. This limits their expressiveness. For instance in the experiment of Section 5.4, MIG attributes the same compactness score (0.0) to representations where a factor corresponds to two and four code dimensions. We believe that if these limitations were addressed, information-based metrics would be more interesting solutions.

6.3 Reporting Results

Disentanglement properties should be measured separately. We share this opinion with [5] and [6]. In our experiments, we showed we could vary properties independently and get the same overall score in very different situations. Metrics measuring all at once make the analysis and comparison of algorithms imprecise. This is particularly true in cases where a parameter balances reconstruction error and factor separation (e.g. $\beta-$VAE [1]). Using a single metric to measure both explicitness and modularity makes it impossible to determine the contribution of each property to the score.

Disentanglement should be measured for each factor independently. While global scores give a quick impression on disentanglement quality, they do not paint the whole picture and can be deceiving. It is impossible to tell from a single number if a model performs generally well except on a few problematic factors, or equally badly on all of them.

The first case might indicate a problem with the data or the choice of factors, while in the second it indicates poor performance of the representation model.

Metrics should be run several times on the same representation. One should report average scores alongside standard deviation. Many metrics implement stochastic components. For instance, intervention-based metrics sample subsets on which they rest their analysis. Predictor-based metrics create validation sets to perform hyper-parameter tuning. Moreover, in applications with large data sets, representations are evaluated on a subset of samples for efficiency. This sampling process adds to the stochasticity of the evaluation even for stable metrics. Performing several measurement runs allows performing statistical significance tests on results to ascertain conclusions from experiments, which should be standard practice when comparing different solutions.

7 Conclusion

In this work we studied how to quantify disentanglement in representations. We conducted an extensive review of supervised disentanglement metrics. We analyzed and compared them experimentally with real-world applications in mind. We reviewed definitions of disentanglement and proposed a new taxonomy organizing the metrics into three families: intervention-based, predictor-based and information-based.

We highlighted the lack of correlation between the different metric scores, and exposed their differences in a series of fully controlled experiments on the robustness to noise, modularity, compactness, hidden factors, calibration and nonlinear relationships. Our experiments revealed different limitations for each metric. We showed how discretization hinders reliability under limited amount of data, noise and nonlinear factor-code relations. We found that predictor-based metrics, when parameterized with caution, were the best performing family of solutions. We discussed the importance of modularity over compactness for practical applications. We concluded, perhaps unsurprisingly, that each disentanglement property should be measured separately for better interpretability.

While we shed some light on the inner working assumption of supervised metrics, several open questions remain. We think that some of the limits exposed in the study can be solved, and thus some metrics, notably from the information-based family, could prove to be stronger solutions than they are now. Also, supervised metrics necessitate factors to be identified and measured which is not always possible when dealing with real-world data. This is why research efforts are now increasingly focused on measuring disentanglement without ground truth factors. This study intentionally left out unsupervised metrics, which is open for future work.

References

[1] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "β-VAE: Learning basic visual concepts with a constrained variational framework," in *International Conference on Learning Representations*, 2017.

[2] H. Kim and A. Mnih, "Disentangling by factorising," in *International Conference on Machine Learning*, 2018.

[3] M. Kim, Y. Wang, P. Sahu, and V. Pavlovic, "Relevance Factor VAE: Learning and identifying disentangled factors," *arXiv:1902.01568*, 2019.

[4] R. Suter, D. Miladinovic, B. Schölkopf, and S. Bauer, "Robustly disentangled causal mechanisms: Validating deep representations for interventional robustness," in *International Conference on Machine Learning*, 2019.

[5] C. Eastwood and C. K. I. Williams, "A framework for the quantitative evaluation of disentangled representations," in *International Conference on Learning Representations*, 2018.

[6] K. Ridgeway and M. C. Mozer, "Learning deep disentangled embeddings with the f-statistic loss," in *Advances in Neural Information Processing Systems*, 2018.

[7] A. Kumar, P. Sattigeri, and A. Balakrishnan, "Variational inference of disentangled latent concepts from unlabeled observations," in *International Conference on Learning Representations*, 2018.

[8] R. T. Q. Chen, X. Li, R. B. Grosse, and D. K. Duvenaud, "Isolating sources of disentanglement in variational autoencoders," in *Advances in Neural Information Processing Systems*, 2018.

[9] K. Do and T. Tran, "Theory and evaluation metrics for learning disentangled representations," in *International Conference on Learning Representations*, 2020.

[10] Z. Li, J. V. Murkute, P. K. Gyawali, and L. Wang, "Progressive learning and disentanglement of hierarchical representations," in *International Conference on Learning Representations*, 2020.

[11] A. Sepliarskaia, J. Kiseleva, and M. de Rijke, "Evaluating disentangled representations," *arXiv:1910.05587*, 2020.

[12] S. Duan, L. Matthey, A. Saraiva, N. Watters, C. Burgess, A. Lerchner, and I. Higgins, "Unsupervised model selection for variational disentangled representation learning," in *International Conference on Learning Representations*, 2020.

[13] X. Liu, S. Thermos, G. Valvano, A. Chartsias, A. O'Neil, and S. A. Tsaftaris, "Metrics for exposing the biases of content-style disentanglement," *arXiv:2008.12378*, 2020.

[14] K. Ridgeway, "A survey of inductive biases for factorial representation-learning," *arXiv:1612.05299*, 2016.

[15] F. Locatello, M. Tschannen, S. Bauer, G. Rätsch, B. Schölkopf, and O. Bachem, "Disentangling factors of variations using few labels," in *International Conference on Learning Representations*, 2020.

[16] F. Locatello, S. Bauer, M. Lucic, G. Raetsch, S. Gelly, B. Schölkopf, and O. Bachem, "Challenging common assumptions in the unsupervised learning of disentangled representations," in *International Conference on Machine Learning*, 2019.

[17] F. Locatello, G. Abbati, T. Rainforth, S. Bauer, B. Schölkopf, and O. Bachem, "On the fairness of disentangled representations," in *Advances in Neural Information Processing Systems*, 2019.

[18] S. van Steenkiste, F. Locatello, J. Schmidhuber, and O. Bachem, "Are disentangled representations helpful for abstract visual reasoning?," in *Advances in Neural Information Processing Systems*, 2019.

[19] G. Desjardins, A. Courville, and Y. Bengio, "Disentangling factors of variation via generative entangling," *arXiv:1210.5474*, 2012.

[20] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets," in *Advances in Neural Information Processing Systems*, 2016.

[21] G. Boulianne, "A study of inductive biases for unsupervised speech representation learning," *IEEE Transactions on Audio, Speech and Language Processing*, vol. 28, pp. 2781–2795, Oct. 2020.

[22] C. P. Burgess, I. Higgins, A. Pal, L. Matthey, N. Watters, G. Desjardins, and A. Lerchner, "Understanding disentangling in β-vae," in *Workshop on Learning Disentangled Representations at the 31st Conference on Neural Information Processing Systems*, 2017.

[23] E. Mathieu, T. Rainforth, N. Siddharth, and Y. W. Teh, "Disentangling disentanglement in variational autoencoders," in *36th International Conference on Machine Learning*, 2019.

[24] V. Thomas, E. Bengio, W. Fedus, J. Pondard, P. Beaudoin, H. Larochelle, J. Pineau, D. Precup, and Y. Bengio, "Disentangling the independently controllable factors of variation by interacting with the world," in *Workshop on Learning Disentangled Representations at the 31st Conference on Neural Information Processing Systems*, 2017.

[25] O. Press, T. Galanti, S. Benaim, and L. Wolf, "Emerging disentanglement in auto-encoder based unsupervised image content transfer," in *International Conference on Learning Representations*, 2019.

[26] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, pp. 1798–1828, Aug. 2013.

[27] B. Schölkopf, D. Janzing, J. Peters, E. Sgouritsa, K. Zhang, and J. Mooij, "On causal and anticausal learning," in *International Conference on Machine Learning*, 2012.

[28] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *arXiv:2004.07780*, 2020.

[29] A. H. Abdi, P. Abolmaesumi, and S. Fels, "A preliminary study of disentanglement with insights on the inadequacy of metrics," *arXiv:1911.11791*, 2019.

[30] J. Schmidhuber, "Learning Factorial Codes By Predictability Minimization," *Neural Computation*, vol. 4, no. 6, pp. 863–879, 1992.

[31] D. Bouchacourt, R. Tomioka, and S. Nowozin, "Multi-level variational autoencoder: Learning disentangled representations from grouped observations," in *AAAI*, 2018.

[32] J. Peters, D. Janzing, and B. Schölkopf, *Elements of Causal Inference: Foundations and Learning Algorithms*. Cambridge, MA, USA: MIT Press, 2017.

[33] B. Esmaeili, H. Wu, S. Jain, A. Bozkurt, N. Siddharth, B. Paige, D. H. Brooks, J. Dy, and J.-W. van de Meent, "Structured disentangled representations," in *International Conference on Artificial Intelligence and Statistics*, 2019.

[34] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[35] L. Paninski, "Estimation of entropy and mutual information," *Neural Computation*, vol. 15, no. 6, pp. 1191–1253, 2003.

[36] J. Hausser and K. Strimmer, "Entropy inference and the james-stein estimator, with application to nonlinear gene association networks," *Journal of Machine Learning Research*, vol. 10, no. 50, pp. 1469–1484, 2009.

[37] S. E. Reed, Y. Zhang, Y. Zhang, and H. Lee, "Deep visual analogy-making," in *Advances in Neural Information Processing Systems*, 2015.

[38] Y. LeCun, Fu Jie Huang, and L. Bottou, "Learning methods for generic object recognition with invariance to pose and lighting," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2004.

[39] M. G. Kendall, *Rank correlation methods.* Charles Griffin and Co. Ltd., London, 1948.

[40] W.-N. Hsu, Y. Zhang, R. J. Weiss, H. Zen, Y. Wu, Y. Wang, Y. Cao, Y. Jia, Z. Chen, J. Shen, P. Nguyen, and R. Pang, "Hierarchical generative modeling for controllable speech synthesis," in *International Conference on Learning Representations*, 2019.

[41] M. Rolínek, D. Zietlow, and G. Martius, "Variational autoencoders pursue pca directions (by accident)," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2019.